Module 3: Linked Lists

A **linked list** is a fundamental data structure in computer science. It mainly allows efficient **insertion** and **deletion** operations compared to arrays. Like arrays, it is also used to implement other data structures like stack, queue and deque. Here's the comparison of Linked List vs Arrays.

Linked List:

- **Data Structure:** Non-contiguous
- **Memory Allocation:** Typically allocated one by one to individual elements
- **Insertion/Deletion:** Efficient
- **Access:** Sequential

Array:

- **Data Structure:** Contiguous
- **Memory Allocation:** Typically allocated to the whole array
- **Insertion/Deletion:** Inefficient
- **Access:** Random

Advantages of Arrays

- **DataAccessingisfaster:** Data can be accessed very efficiently by specifying the array name and corresponding index like A[i]-> to access i^{th} item in an array.
- **Simple:** Arrays are simple to understand and use.

Disadvantages of Arrays:

- **Array size is fixed:** The array size is fixed during compilation time.
- **Insertion and deletion operations involving arrays is tedious job**: Inserting and deleting an item at a given position consumes time.

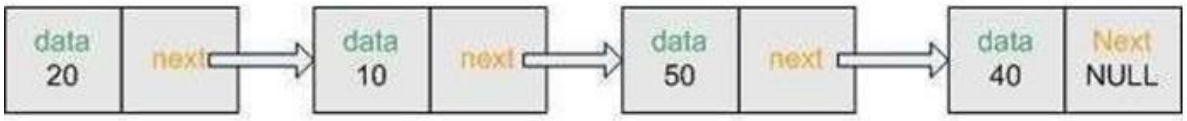
The above disadvantages can be overcome using linked lists.

A linked list is a data structure that is collection of zero or more nodes where each node has some information. If each node in the list has only one link, it is called a **singly linked list**. If it has two links one containing the address of the next node and the other link containing the address of the previous node it is called a **doubly linked list.**

Each node in the singly linked list has two fields namely:

Data->This field is used to store the data or information.

Next(Link)->This field contains the address of the next node.



Linked list

Types of Linked Lists: The 4 types of linked lists are as follows:

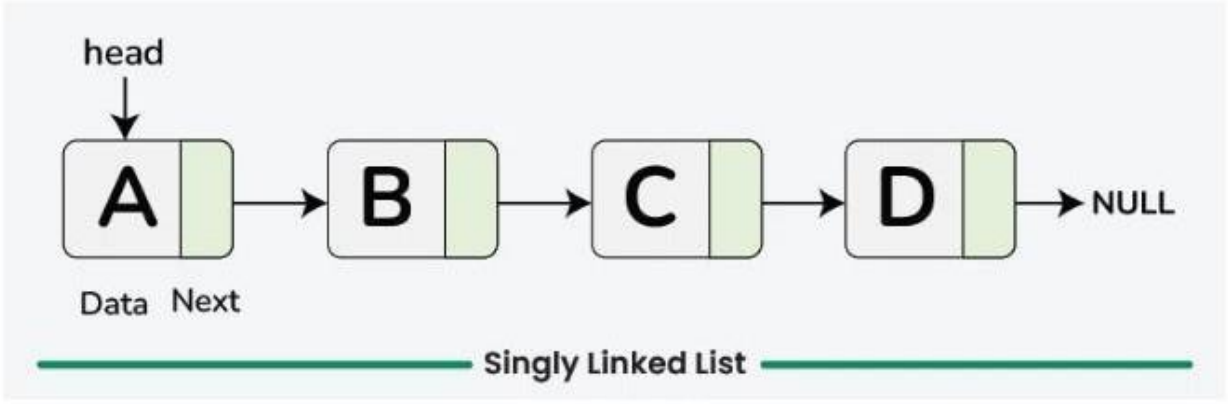
- Singly Linked Lists
- Doubly Linked Lists
- Circular Singly Linked Lists
- Circular Doubly Linked Lists

Singly Linked List

A **singly linked list** is a fundamental data structure in computer science and programming, it consists of **nodes** where each node contains a **data** field and a **reference** to the next node in the node. The last node points to **null**, indicating the end of the list. This linear structure supports efficient insertion and deletion operations, making it widely used in various applications. In this tutorial, we'll explore the node structure, understand the operations on singly linked lists (traversal, searching, length determination, insertion, and deletion), and provide detailed explanations and code examples to implement these operations effectively.

Understanding Node Structure

In a singly linked list, each node consists of two parts: data and a pointer to the next node. The data part stores the actual information, while the pointer (or reference) part stores the address of the next node in the sequence. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.



Singly Linked List

In this representation, each box represents a node, with an arrow indicating the link to the next node. The last node points to NULL, indicating the end of the list.

```
// Definition of a Node in a singly linked list
struct Node
{
    int data;
    struct Node* next;
};

// Function to create a new Node
struct Node* newNode(int data)
{
   struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
   temp->data = data;
   temp->next = NULL;
   return temp;
}
```

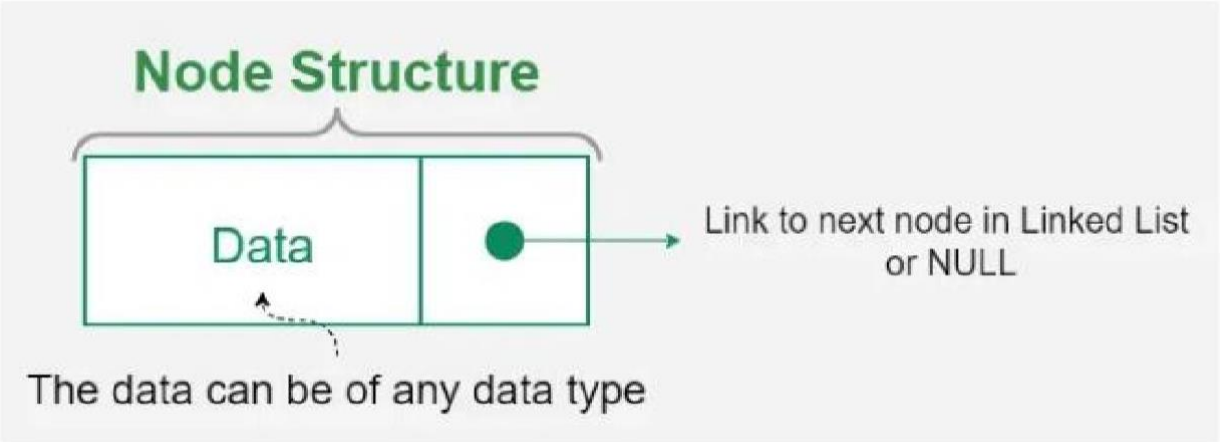
In this example, the Node class contains an integer data field (**data**) to store the information and a pointer to another Node (**next**) to establish the link to the next node in the list.

Node Structure and Representation of Singly Linked List:

Node Structure of Linked List:

A singly linked list consists of nodes, where each node contains:

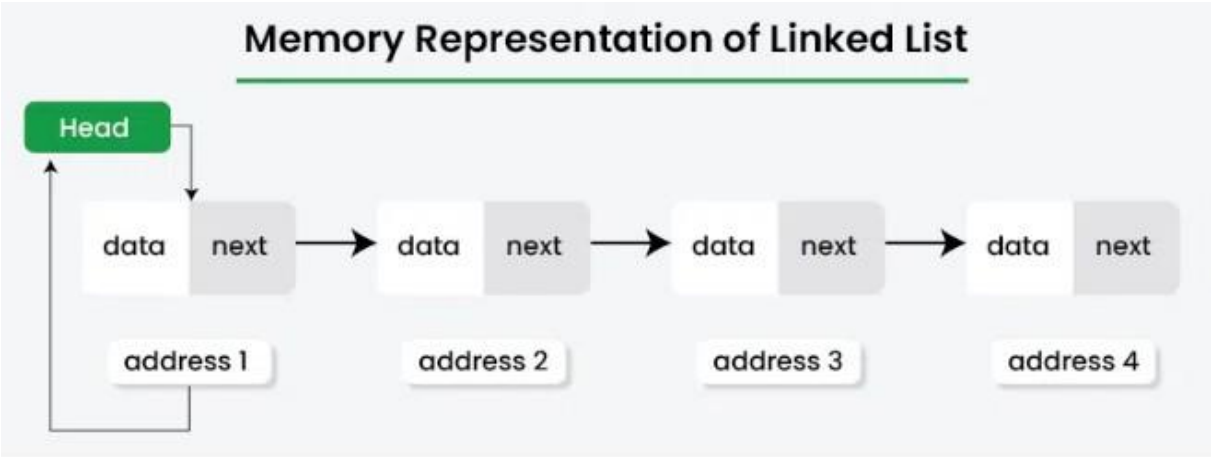
- Data Field: Holds the actual data.
- Next Field: A pointer/reference to the next node.



Node Structure of Singly Linked List

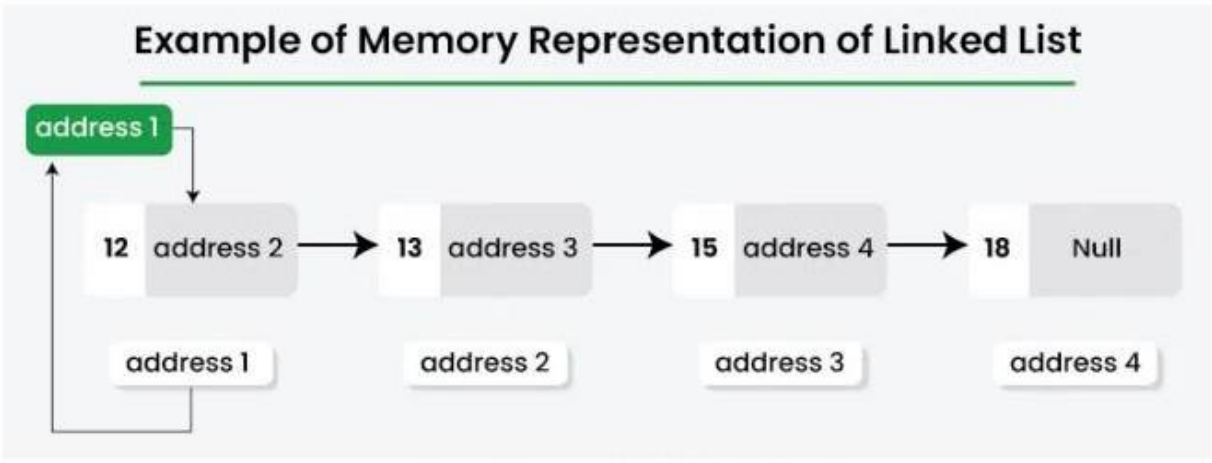
Memory Representation of a Linked List

Unlike arrays, where elements are stored in contiguous memory locations, linked lists allocate memory dynamically for each node. This means that each node can be located anywhere in the memory and they are connected via pointers.



Memory Representation of Singly Linked List

Before looking at the memory representation of singly linked list. Let's first create a linked list having four nodes. Each node has two parts: one part holds a value (data) and the other part holds the address of the next node. The first node is called the head node and it points to the first element in the list. The last node in the list points to NULL, which means there are no more nodes after it.

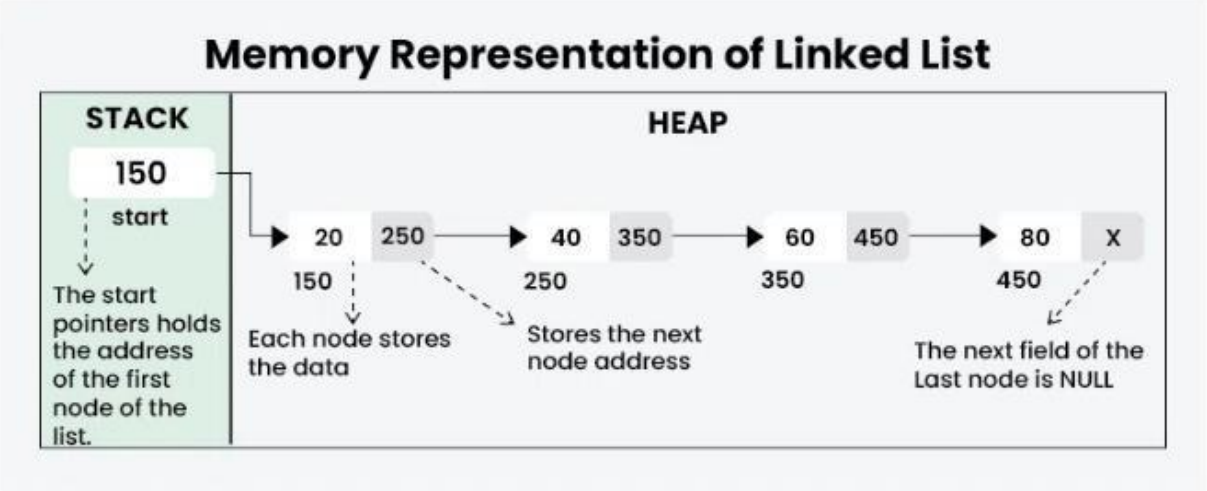


Example of Memory representation of Linked list

In the above singly linked list, each node is connected by pointers. These pointers show the address of the next node, which allowing us to move through the list in forward direction. This connection is shown with arrows in the diagram, making it clear how each node links to the next one.

How is Memory Allocation done for Linked List?

Let's see how memory is allocated for the linked list. The images below illustrate this more clearly.



Memory Allocation for Linked List

- **Heap Memory:** The nodes of the linked list are dynamically allocated, which allocate memory from the heap. Heap memory is used for objects whose size may not be known at **compile time** and allows for **dynamic memory management.**
- **Stack Memory:** The pointer "**head**" is typically defined within a function (like main). Local variables, including pointers, are stored in the **stack**, which has a fixed size and is managed automatically when the function is called and returns.

Operations on Singly Linked List

- **Traversal**
- **Searching**
- **Insertion:**
  - Insert at the beginning
  - Insert at the end
  - Insert at a specific position
- **Deletion:**
  - Delete from the beginning
  - Delete from the end
  - Delete a specific node

Let's go through each of the operations mentioned above, one by one.

Traversal in Singly Linked List**:**

Traversal involves visiting each node in the linked list and performing some operation on the data. A simple traversal function would print or process the data of each node.

Step-by-step approach:

- Initialize a pointer current to the head of the list.
- Use a while loop to iterate through the list until the current pointer reaches NULL.

- Inside the loop, print the data of the current node and move the current pointer to the next node.

Below is the function for traversal in singly Linked List:

```
// Function to traverse and print the elements
// of the linked list
void traverseLinkedList(struct Node* head)
{
   // Start from the head of the linked list
   struct Node* current = head;
   // Traverse the linked list until reaching the end (NULL)
   while (current != NULL)
   {
      // Print the data of the current node
      printf("%d ", current->data);
      // Move to the next node
      current = current->next;
   }
   printf("\n");
}
```

Output

```
1 2 3
```

Searching in Singly Linked List:

Searching in a Singly Linked List refers to the process of looking for a specific element or value within the elements of the linked list.

Step-by-step approach:

1. Traverse the Linked List starting from the head.
2. Check if the current node's data matches the target value.
   - If a match is found, return **true**.
3. Otherwise, Move to the next node and repeat steps 2.
4. If the end of the list is reached without finding a match, return **false**.

Below is the function for searching in singly linked list:

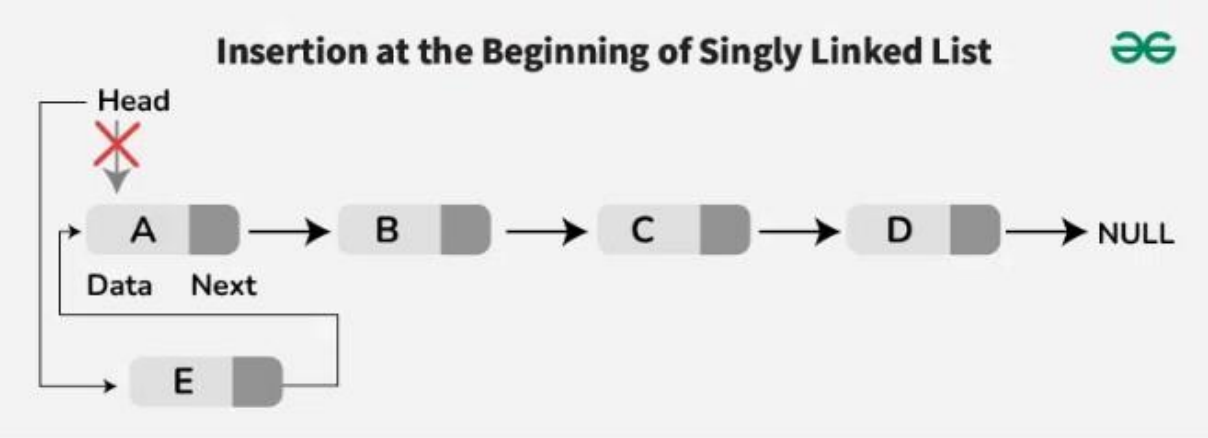
```
/ Function to search for a value in the Linked List
bool searchLinkedList(struct Node* head, int target)
{
   // Traverse the Linked List
   while (head != NULL)
   {
      // Check if the current node's
      // data matches the target value
      if (head->data == target)
      {
```

```
            return true; // Value found
        }
        // Move to the next node
        head = head->next;
    }
    return false; // Value not found
}
```

Insertion in Singly Linked List:

Insertion is a fundamental operation in linked lists that involves adding a new node to the list. There are several scenarios for insertion:

a) Insertion at the Beginning of Singly Linked List:



Insert a Node at the Front/Beginning of Linked List

Step-by-step approach:

- Create a new node with the given value.
- Set the **next** pointer of the new node to the current head.
- Move the head to point to the new node.
- Return the new head of the linked list.

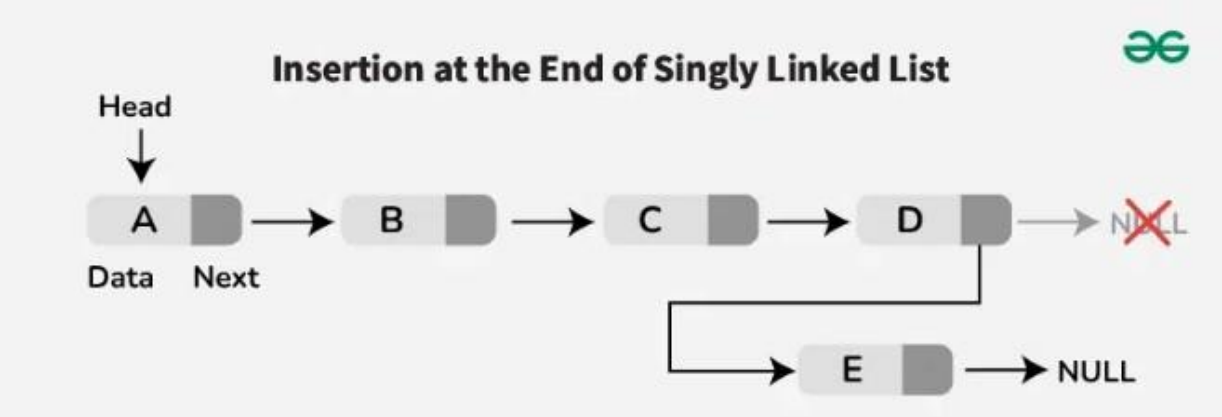
Below is the function for insertion at the beginning of singly linked list:

```
// Function to insert a new node at the beginning of the linked list
struct Node* insertAtBeginning(struct Node* head, int value)
{
    // Create a new node with the given value
    struct Node* new_node = newNode(value);
    // Set the next pointer of the new node to the current head
    new_node->next = head;
    // Move the head to point to the new node
    head = new_node;
    // Return the new head of the linked list
```

```
    return head;
}
```

b) Insertion at the End of Singly Linked List:

To insert a node at the end of the list, traverse the list until the last node is reached, and then link the new node to the current last node-



Insertion at end of Linked List

Step-by-step approach:

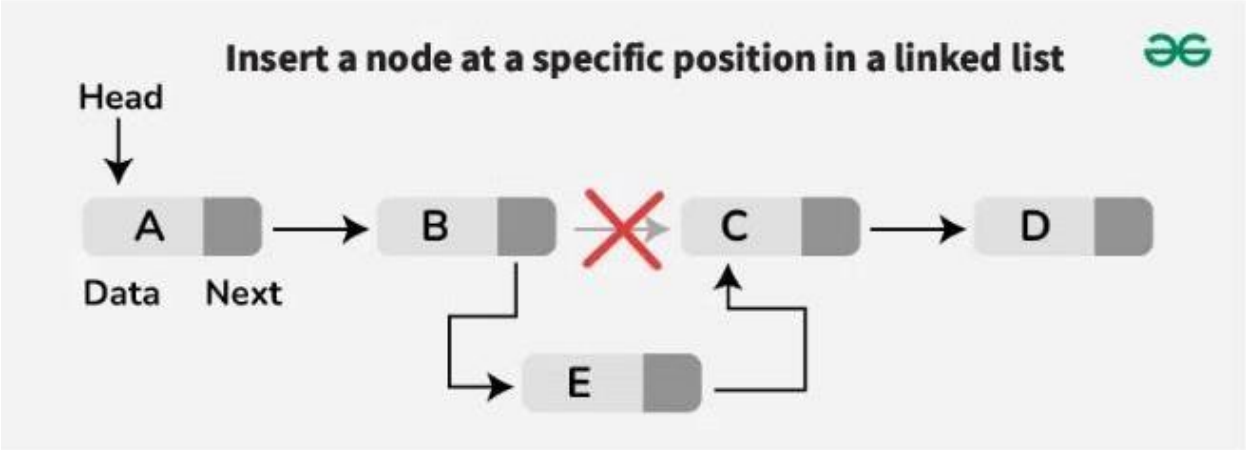
- Create a new node with the given value.
- Check if the list is empty:
  - If it is, make the new node the head and return.
- Traverse the list until the last node is reached.
- Link the new node to the current last node by setting the last node's next pointer to the new node.

Below is the function for insertion at the end of singly linked list:

```
// Function to insert a node at the end of the linked list
struct Node* insertAtEnd(struct Node* head, int value)
{
    // Create a new node with the given value
    struct Node* new_node = newNode(value);
    // If the list is empty, make the new node the head
    if (head == NULL)
        return new_node;
    // Traverse the list until the last node is reached
    struct Node* curr = head;
    while (curr->next != NULL) {
        curr = curr->next;
    }
    // Link the new node to the current last node
    curr->next = new_node;
    return head;
}
```

c. Insertion at a Specific Position of the Singly Linked List:

To insert a node at a specific position, traverse the list to the desired position, link the new node to the next node, and update the links accordingly.



We mainly find the node after which we need to insert the new node. If we encounter a NULL before reaching that node, it means that the given position is invalid.
Below is the function for insertion at a specific position of the singly linked list:

```
// Function to insert a node at a specified position
struct Node* insertPos(struct Node* head, int pos, int data)
{
    if (pos < 1)
    {
        printf("Invalid position!\n");
        return head;
    }
    // Special case for inserting at the head
    if (pos == 1) {
        struct Node* temp = getNode(data);
        temp->next = head;
        return temp;
    }
    // Traverse the list to find the node
    // before the insertion point
    struct Node* prev = head;
    int count = 1;
    while (count < pos - 1 && prev != NULL)
    {
        prev = prev->next;
        count++;
    }
    // If position is greater than the number of nodes
    if (prev == NULL) {
        printf("Invalid position!\n");
        return head;
    }
    // Insert the new node at the specified position
    struct Node* temp = getNode(data);
```

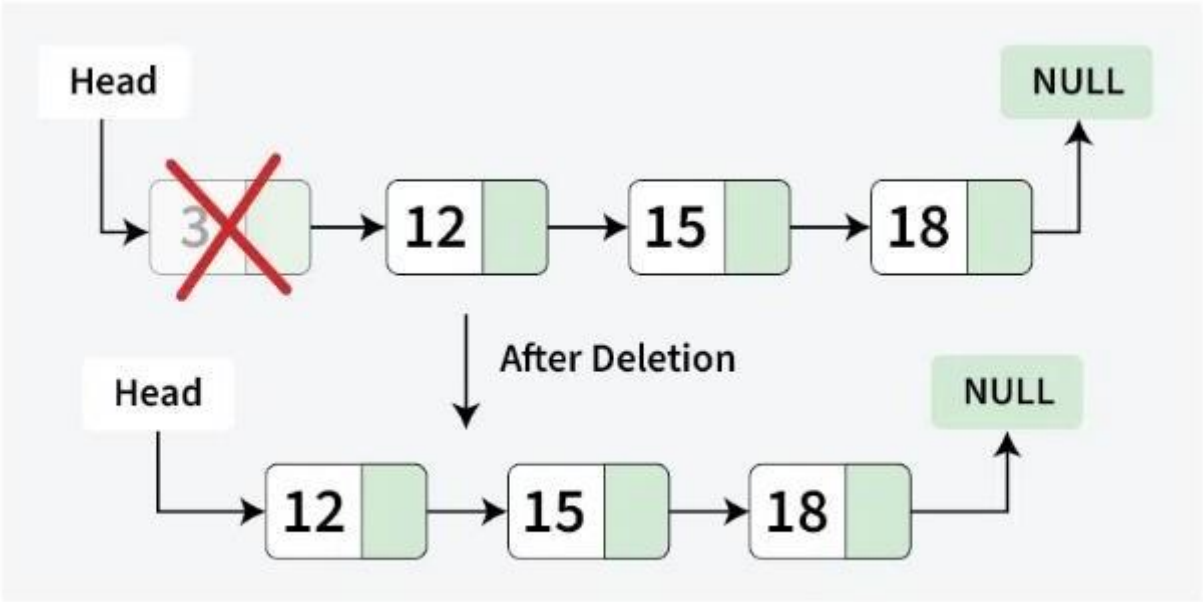
```
    temp->next = prev->next;
    prev->next = temp;
    return head;
}
```

Deletion in Singly Linked List:

Deletion involves removing a node from the linked list. Similar to insertion, there are different scenarios for deletion:

a) Deletion at the Beginning of Singly Linked List:

To delete the first node, update the head to point to the second node in the list.



Deletion at beginning in a Linked List

Steps-by-step approach:

- Check if the head is **NULL**.
    - If it is, return **NULL** (the list is empty).
- Store the current head node in a temporary variable **temp**.
- Move the head pointer to the next node.
- Delete the temporary node.
- Return the new head of the linked list.

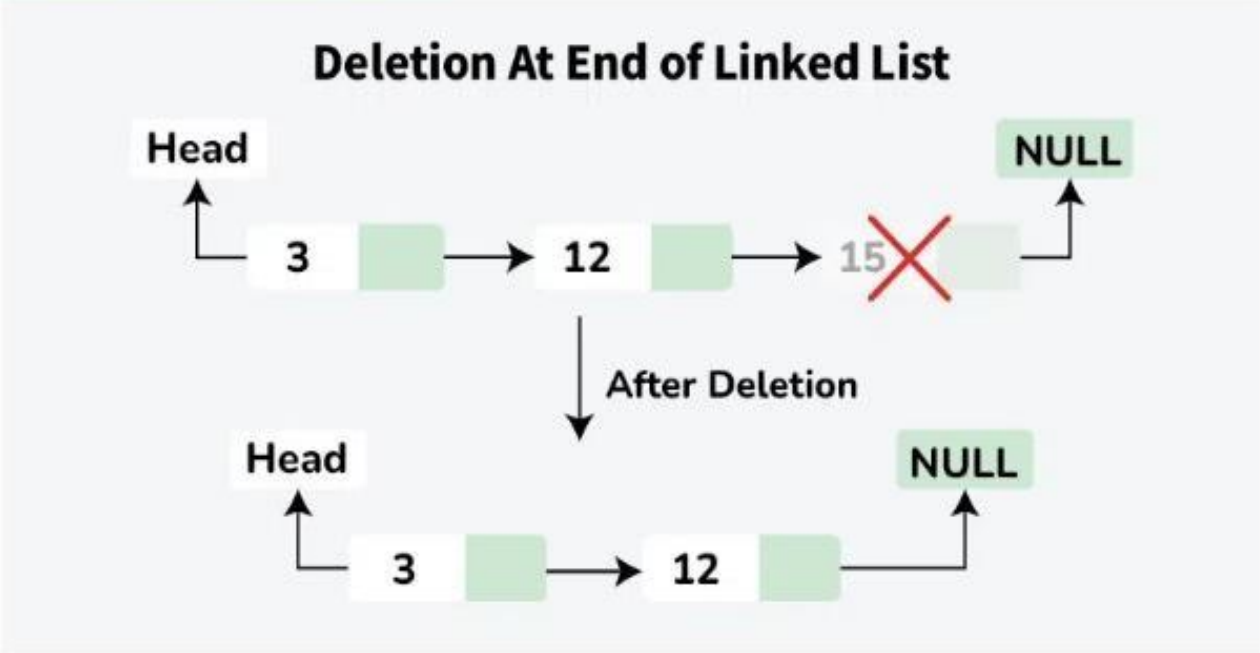
Below is the function for deletion at the beginning of singly linked list:

```
// Function to remove the first node of the linked list
struct Node* removeFirstNode(struct Node* head)
{
   if (head == NULL)
      return NULL;
```

```
    // Move the head pointer to the next node
    struct Node* temp = head;
    head = head->next;
    // Free the memory of the old head
    free(temp);
    return head;
}
```

b) Deletion at the End of Singly Linked List:

To delete the last node, traverse the list until the second-to-last node and update its next field to None.



Deletion at the end of linked list

Step-by-step approach:

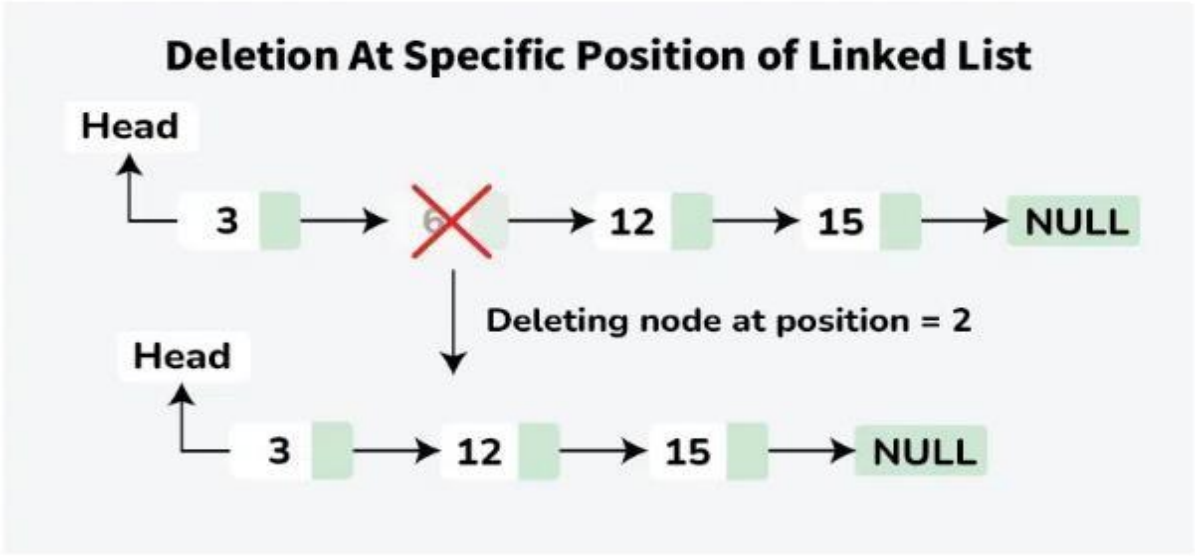
- Check if the head is **NULL**.
  - If it is, return NULL (the list is empty).
- Check if the head's **next** is **NULL** (only one node in the list).
  - If true, delete the head and return **NULL**.
- Traverse the list to find the second last node (**second_last**).
- Delete the last node (the node after **second_last**).
- Set the **next** pointer of the second last node to **NULL**.
- Return the head of the linked list.

Below is the function for deletion at the end of singly linked list:

```c
// Function to remove the last node of the linked list
struct Node* removeLastNode(struct Node* head)
{
   if (head == NULL)
      return NULL;

   if (head->next == NULL) {
      free(head);
      return NULL;
   }

   // Find the second last node
   struct Node* second_last = head;
   while (second_last->next->next != NULL)
      second_last = second_last->next;

   // Delete last node
   free(second_last->next);

   // Change next of second last
   second_last->next = NULL;

   return head;
}
```

c. Deletion at a Specific Position of Singly Linked List:

To delete a node at a specific position, traverse the list to the desired position, update the links to bypass the node to be deleted.



Delete a Linked List node at a given position

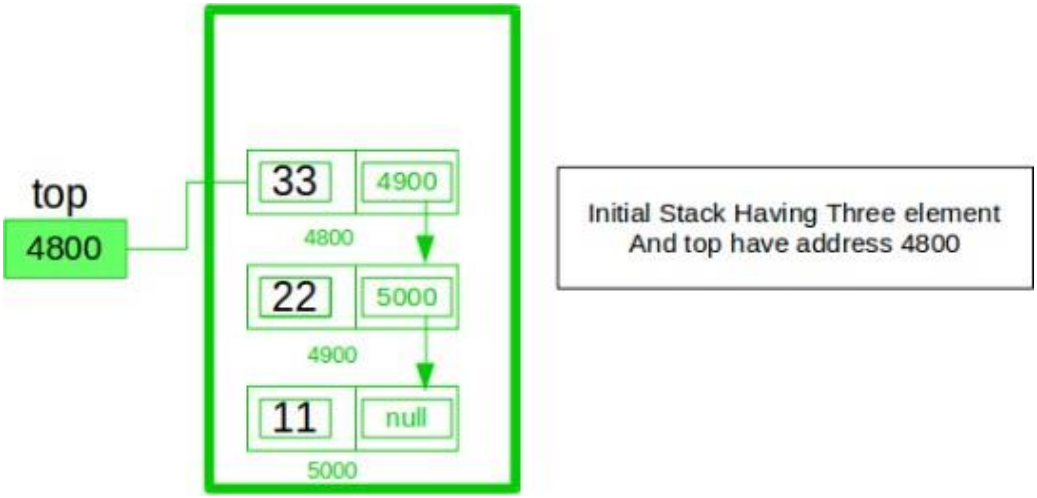
Step-by-step approach:

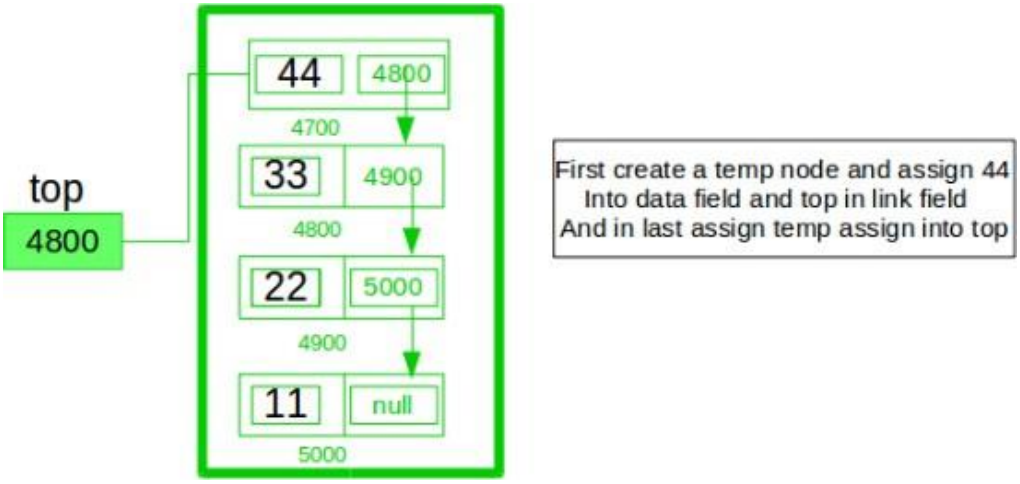
- Check if the list is empty or the position is invalid, return if so.
- If the head needs to be deleted, update the head and delete the node.
- Traverse to the node before the position to be deleted.
- If the position is out of range, return.
- Store the node to be deleted.
- Update the links to bypass the node.
- Delete the stored node.

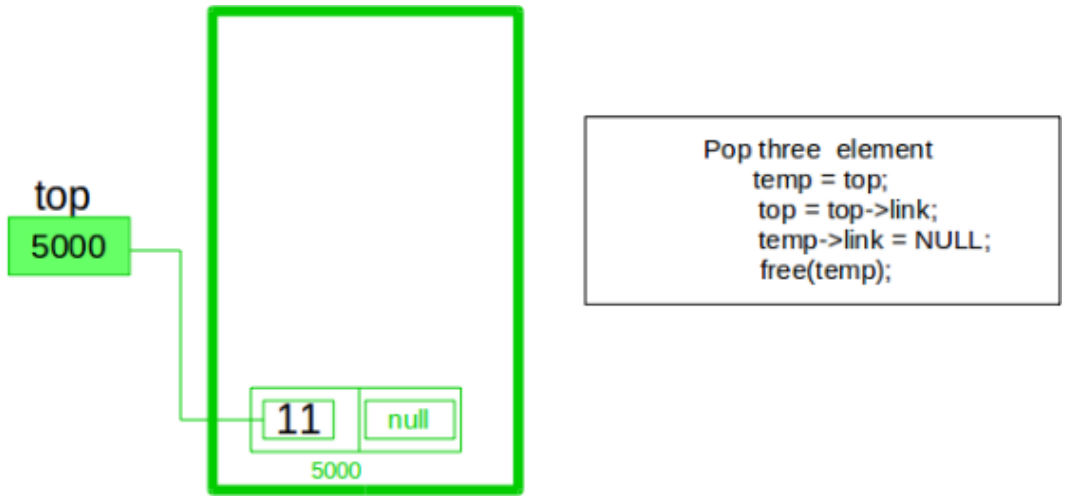
```
// Function to delete a node at a specific position
struct Node* deleteAtPosition(struct Node* head, int position)
{
   // If the list is empty or the position is invalid
   if (head == NULL || position < 1) {
      return head;
   }
   // If the head needs to be deleted
   if (position == 1) {
      struct Node* temp = head;
      head = head->next;
      free(temp);
      return head;
   }
   // Traverse to the node before the position to be deleted
   struct Node* curr = head;
   for (int i = 1; i < position - 1 && curr != NULL; i++) {
      curr = curr->next;
   }
   // If the position is out of range
   if (curr == NULL || curr->next == NULL) {
      return head;
   }
   // Store the node to be deleted
   struct Node* temp = curr->next;

   // Update the links to bypass the node to be deleted
   curr->next = curr->next->next;
   // Delete the node
   free(temp);
   return head;
}
```

Implement a stack using singly linked list:

To implement a stack using the singly linked list concept, all the singly linked list operations should be performed based on Stack operations LIFO(last in first out) and with the help of that knowledge, we are going to implement a stack using a singly linked list.
So we need to follow a simple rule in the implementation of a stack which is **last in first out** and all the operations can be performed with the help of a top variable. Let us learn how to perform **Pop, Push, Peek, and Display** operations in the following







In the stack Implementation, a stack contains a top pointer. which is the "head" of the stack where pushing and popping items happens at the head of the list. The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the "top" pointer.
The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed. Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow. Here each new node will be dynamically allocated. so overflow is not possible.

Stack Operations:

- **push():** Insert a new element into the stack i.e just insert a new element at the beginning of the linked list.
- **pop():** Return the top element of the Stack i.e simply delete the first element from the linked list.
- **peek():** Return the top element.
- **display():** Print all elements in Stack.

Push Operation:

- *Initialise a node*
- *Update the value of that node by data i.e.* ***node->data = data***
- *Now link this node to the top of the linked list*
- *And update top pointer to the current node*

Pop Operation:

- *First Check whether there is any node present in the linked list or not, if not then return*
- *Otherwise make pointer let say* ***temp*** *to the top node and move forward the top node by 1 step*
- *Now free this temp node*

Peek Operation:

- *Check if there is any node present or not, if not then return.*
- *Otherwise return the value of top node of the linked list*

Display Operation:

- *Take a **temp** node and initialize it with top pointer*
- *Now start traversing temp till it encounters NULL*
- *Simultaneously print the value of the temp node*

```
// C program to implement a stack using singly linked list
#include <limits.h>
#include  <stdio.h>
#include <stdlib.h>
// Struct representing a node in the linked list
typedef struct Node
{
   int data;
   struct Node* next;
} Node;
Node* createNode(int new_data)
{
   Node* new_node = (Node*)malloc(sizeof(Node));
   new_node->data = new_data;
   new_node->next = NULL;
   return new_node;
}
// Struct to implement stack using a singly linked list
typedef struct Stack
{
   Node* head;
} Stack;
// Constructor to initialize the stack
void initializeStack(Stack* stack)
{
            stack->head = NULL;
}
// Function to check if the stack is empty
int isEmpty(Stack* stack)
{
    // If head is NULL, the stack is empty
   return stack->head == NULL;
}
// Function to push an element onto the stack
void push(Stack* stack, int new_data)
```

```
{
    // Create a new node with given data
    Node* new_node = createNode(new_data);
    // Check if memory allocation for the new node failed
    if (!new_node)
    {
        printf("\nStack Overflow");
        return;
    }
    // Link the new node to the current top node
    new_node->next = stack->head;
    // Update the top to the new node
    stack->head = new_node;
}
// Function to remove the top element from the stack
void pop(Stack* stack)
{
    // Check for stack underflow
    if (isEmpty(stack))
    {
        printf("\nStack Underflow\n");
        return;
    }
    else
    {
        // Assign the current top to a temporary variable
        Node* temp = stack->head;

        // Update the top to the next node
        stack->head = stack->head->next;
        // Deallocate the memory of the old top node
        free(temp);
    }
}
// Function to return the top element of the stack
int peek(Stack* stack)
{
    // If stack is not empty, return the top element
    if (!isEmpty(stack))
        return stack->head->data;
    else
    {
        printf("\nStack is empty");
        return INT_MIN;
```

```
    }
}
// Driver program to test the stack implementation
int main()
{
    // Creating a stack
    Stack stack;
    initializeStack(&stack);
    // Push elements onto the stack
    push(&stack, 11);
    push(&stack, 22);
    push(&stack, 33);
    push(&stack, 44);
    // Print top element of the stack
    printf("Top element is %d\n", peek(&stack));
    // removing two elemements from the top
    printf("Removing two elements...\n");
    pop(&stack);
    pop(&stack);
    // Print top element of the stack
    printf("Top element is %d\n", peek(&stack));
    return 0;
}
```

Output

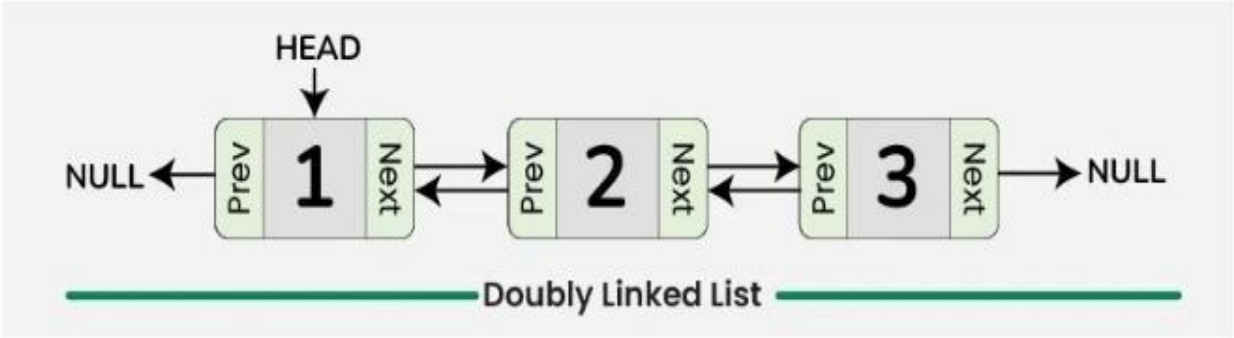
```
Top element is 44

Top element is 22
```

Doubly Linked List

A **doubly linked list** is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.

What is a Doubly Linked List?

A **doubly linked list** is a data structure that consists of a set of nodes, each of which contains a **value** and **two pointers**, one pointing to the **previous node** in the list and one pointing to the **next node** in the list. This allows for efficient traversal of the list in **both directions**, making it suitable for applications where frequent **insertions** and **deletions** are required.

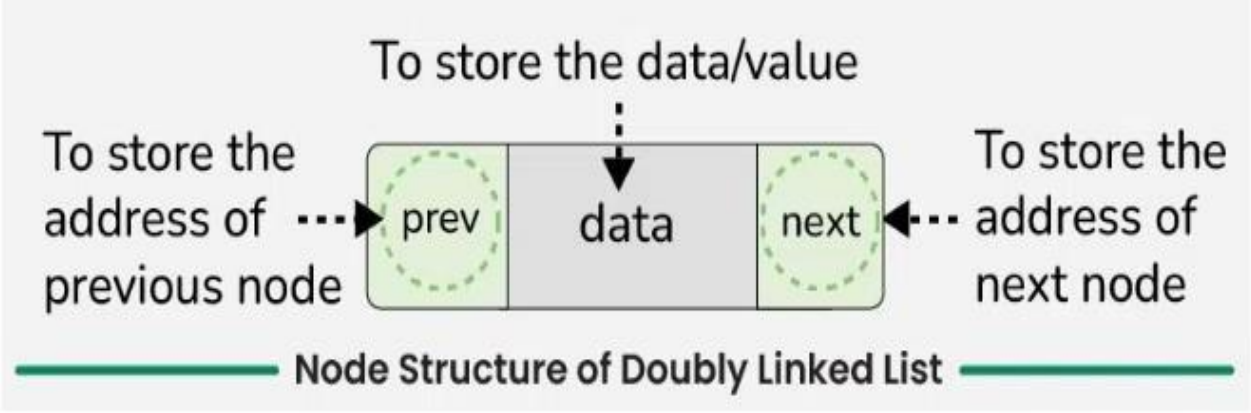


Doubly Linked List

Representation of Doubly Linked List in Data Structure

In a data structure, a doubly linked list is represented using nodes that have three fields:

1. Data
2. A pointer to the next node (**next**)
3. A pointer to the previous node (**prev**)



Node Structure of Doubly Linked List

Node Definition

Here is how a node in a Doubly Linked List is typically represented:

```
struct Node
{
   // To store the Value or data.
   int data;
   // Pointer to point the Previous Element
   Node* prev;
   // Pointer to point the Next Element
   Node* next;
};
```

```
// Function to create a new node
struct Node *createNode(int new_data)
{
   struct Node *new_node = (struct Node *)
   malloc(sizeof(struct Node));
   new_node->data = new_data;
   new_node->next = NULL;
   new_node->prev = NULL;
   return new_node;
}
```

Each node in a **Doubly Linked List** contains the **data** it holds, a pointer to the **next** node in the list, and a pointer to the **previous** node in the list. By linking these nodes together through the **next** and **prev** pointers, we can traverse the list in both directions (forward and backward), which is a key feature of a Doubly Linked List.

Operations on Doubly Linked List

1. Traversal in Doubly Linked List
2. Insertion in Doubly Linked List:
    - Insertion at the beginning of Doubly Linked List
    - Insertion at the end of the Doubly Linked List
    - Insertion at the specific position of the Doubly Linked List
3. Deletion in Doubly Linked List:
    - Deletion of a node at the beginning of Doubly Linked List
    - Deletion of a node at the end of Doubly Linked List
    - Deletion of a node at the specific position of Doubly Linked List

Let's go through each of the operations mentioned above, one by one.

1. Traversal in Doubly Linked List

To Traverse the doubly list, we can use the following steps:

a. Forward Traversal:

- Initialize a pointer to the head of the linked list.
- While the pointer is not null:
    - Visit the data at the current node.
    - Move the pointer to the next node.

```
// Function to traverse the doubly linked list
// in forward direction
void forwardTraversal(struct Node* head)
{
   // Start traversal from the head of the list
   struct Node* curr = head;
```

```
   // Continue until the current node is not
   // null (end of list)
   while (curr != NULL)
{
       // Output data of the current node
      printf("%d ", curr->data);
      // Move to the next node
      curr = curr->next;
   }
   // Print newline after traversal
   printf("\n");
}
```

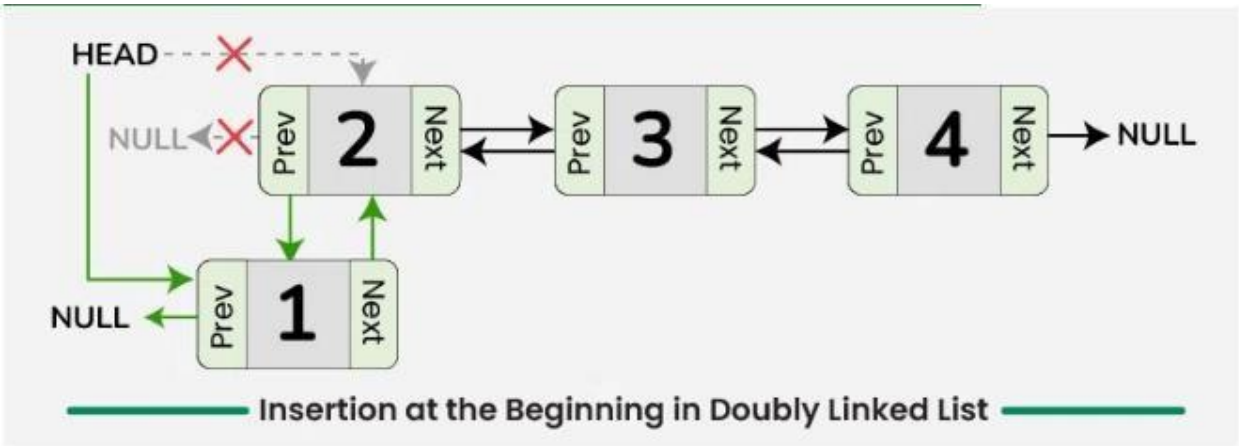
b. Backward Traversal:

- Initialize a pointer to the tail of the linked list.
- While the pointer is not null:
  - Visit the data at the current node.
  - Move the pointer to the previous node.

```
// Function to traverse the doubly linked list
// in backward direction
void backwardTraversal(struct Node* tail)
{
   // Start traversal from the tail of the list
   struct Node* curr = tail;
   // Continue until the current node is not
   // null (end of list)
   while (curr != NULL)
  {
       // Output data of the current node
      printf("%d ", curr->data);
      // Move to the previous node
      curr = curr->prev;
   }
   // Print newline after traversal
   printf("\n");
}
```

2. Insertion in Doubly Linked List:

a. Insertion at the Beginning in Doubly Linked List



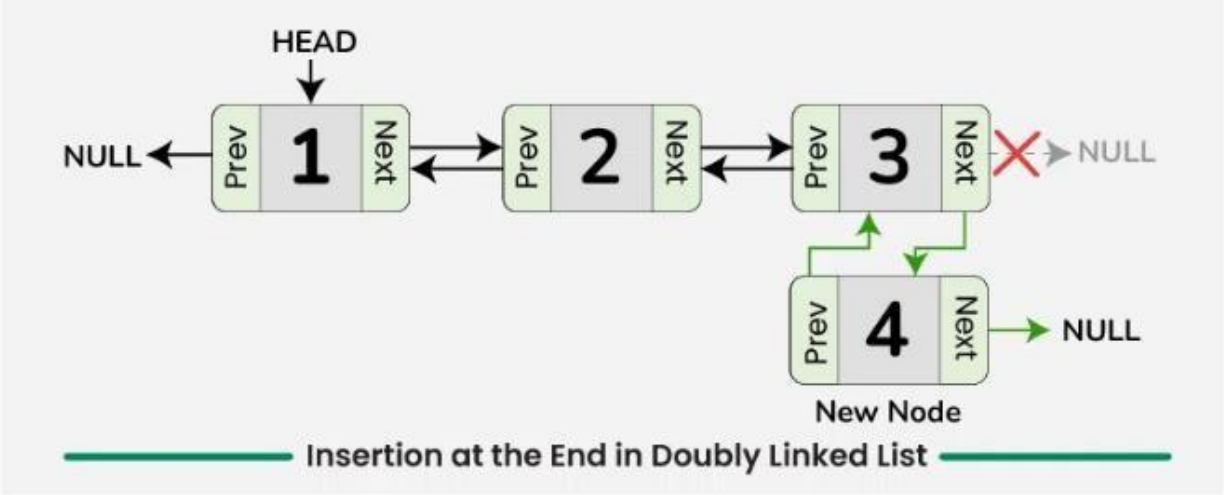
Insertion at the Beginning in Doubly Linked List

To insert a new node at the beginning of the doubly list, we can use the following steps:

- Create a new node, say **new_node** with the given data and set its previous pointer to null, **new_node->prev = NULL**.
- Set the next pointer of new_node to current head, **new_node->next = head.**
- If the linked list is not empty, update the previous pointer of the current head to new_node, **head->prev = new_node**.
- Return new_node as the head of the updated linked list.

```
// Insert a node at the beginning
struct Node* insertBegin(struct Node* head, int data)
{
   // Create a new node
   struct Node* new_node = createNode(data);
    // Make next of it as head
   new_node->next = head;
    // Set previous of head as new node
   if (head != NULL)
   {
      head->prev = new_node;
   }
      // Return new node as new head
   return new_node;
}
```

b. Insertion at the End of Doubly Linked List



Insertion at the End in the Doubly Linked List

To insert a new node at the end of the doubly linked list, we can use the following steps:

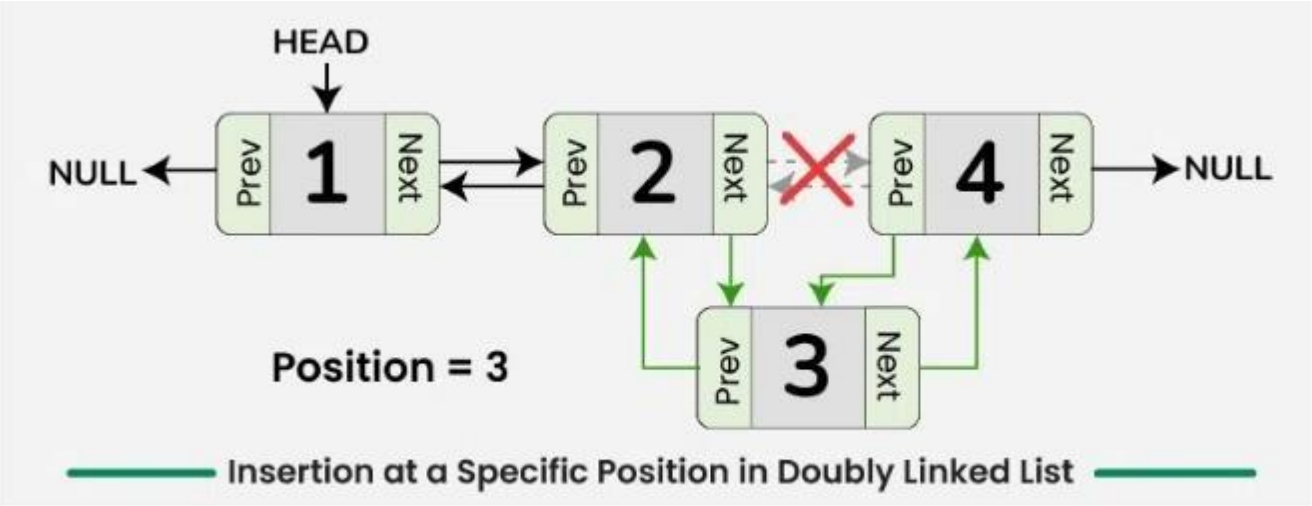
- Allocate memory for a new node and assign the provided value to its data field.
- Initialize the next pointer of the new node to nullptr.
- If the list is empty:
  - Set the previous pointer of the new node to nullptr.
  - Update the head pointer to point to the new node.
- If the list is not empty:
  - Traverse the list starting from the head to reach the last node.
  - Set the next pointer of the last node to point to the new node.
  - Set the previous pointer of the new node to point to the last node.

```
// Function to insert a new node at the end of the
//doubly linked list
struct Node* insertEnd(struct Node *head, int new_data) {
  struct Node *new_node = createNode(new_data);

  // If the linked list is empty, set the
        //new node as the head
  if (head == NULL) {
    head = new_node;
  } else {
    struct Node *curr = head;
    while (curr->next != NULL) {
      curr = curr->next;
    }

    // Set the next of last node to new node
    curr->next = new_node;
```

```
    // Set prev of new node to last node
    new_node->prev = curr;
  }

  return head;
}
```

c. Insertion at the specific position of the Doubly Linked List

To insert a node at a specific Position in doubly linked list, we can use the following steps:



Insertion at a Specific Position in Doubly Linked List

Insertion at a Specific Position in Doubly Linked List

To insert a new node at a specific position,

- If position = 1, create a new node and make it the head of the linked list and return it.
- Otherwise, traverse the list to reach the node at position – 1, say **curr**.
- If the position is valid, create a new node with given data, say **new_node**.
- Update the next pointer of new node to the next of current node and prev pointer of new node to current node, **new_node->next = curr->next** and **new_node->prev = curr.**
- Similarly, update next pointer of current node to the new node, **curr->next = new_node**.
- If the new node is not the last node, update prev pointer of new node's next to the new node, **new_node->next->prev = new_node.**

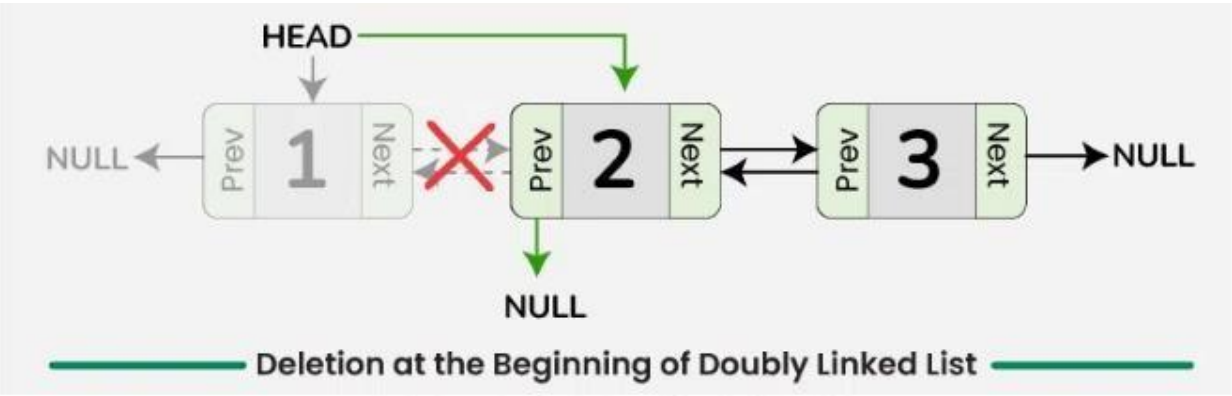
Below is the implementation of the above approach:

```
// Function to insert a new node at a given position
struct Node * insertAtPosition(struct Node * head, int pos, int new_data) {
    // Create a new node
    struct Node * new_node = createNode(new_data);
    // Insertion at the beginning
    if (pos == 1)
    {
        new_node -> next = head;
        // If the linked list is not empty, set the
      //prev of head to new node
        if (head != NULL) {
            head -> prev = new_node;
```

```
        }
        // Set the new node as the head of linked list
        head = new_node;
        return head;
    }
    struct Node * curr = head;
    // Traverse the list to find the node before the insertion point
    for (int i = 1; i < pos - 1 && curr != NULL; ++i) {
        curr = curr -> next;
    }
    // If the position is out of bounds
    if (curr == NULL)
    {
        printf("Position is out of bounds.\n");
        free(new_node);
        return head;
    }
    // Set the prev of new node to curr
    new_node -> prev = curr;
    // Set the next of new node to next of curr
    new_node -> next = curr -> next;
    // Update the next of current node to new node
    curr -> next = new_node;
    // If the new node is not the last node, update
         //the prev of next node to new node
    if (new_node -> next != NULL)
    {
        new_node -> next -> prev = new_node;
    }
    // Return the head of the doubly linked list
    return head;
}
```

3. Deletion in Doubly Linked List

a. Deletion at the Beginning of Doubly Linked List



Deletion at the Beginning of Doubly Linked List

To delete a node at the beginning in doubly linked list, we can use the following steps:

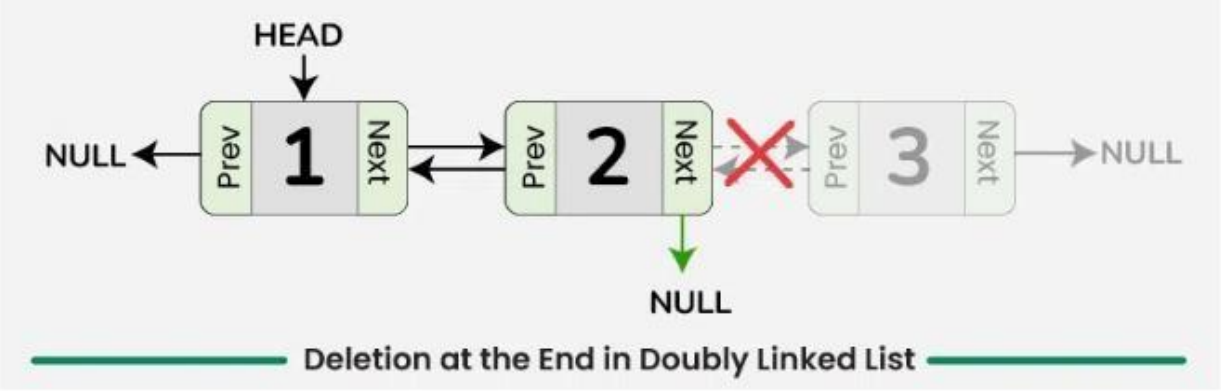
- Check if the list is empty, there is nothing to delete. Return.

- Store the head pointer in a variable, say **temp**.
- Update the head of linked list to the node next to the current head, **head = head->next**.
- If the new head is not NULL, update the previous pointer of new head to NULL, **head->prev = NULL**.

Below is the implementation of the above approach:

```
// Function to delete the first node (head) of the list
// and return the second node as the new head
struct Node *delHead(struct Node *head) {
   // If empty, return NULL
   if (head == NULL)
      return NULL;

   // Store in temp for deletion later
   struct Node *temp = head;

   // Move head to the next node
   head = head->next;

   // Set prev of the new head
   if (head != NULL)
      head->prev = NULL;

   // Free memory and return new head
   free(temp);
   return head;
}
```

b. Deletion at the End of Doubly Linked List



Deletion at the End in Doubly Linked List

To delete a node at the end in doubly linked list, we can use the following steps:

- Check if the doubly linked list is empty. If it is empty, then there is nothing to delete.
- If the list is not empty, then move to the last node of the doubly linked list, say **curr**.

- Update the second-to-last node's next pointer to NULL, **curr->prev->next = NULL**.
- Free the memory allocated for the node that was deleted.

Below is the implementation of the above approach:

```
// Function to delete the last node of the
//doubly linked list
struct Node* delLast(struct Node *head) {

   // Corner cases
   if (head == NULL)
      return NULL;
   if (head->next == NULL) {
      free(head);
      return NULL;
   }

   // Traverse to the last node
   struct Node *curr = head;
   while (curr->next != NULL)
      curr = curr->next;

   // Update the previous node's next pointer
   curr->prev->next = NULL;

   // Delete the last node
   free(curr);

   // Return the updated head
   return head;
}
```

c. Deletion at a Specific Position in Doubly Linked List

Deletion at a Specific Position in Doubly Linked List

To delete a node at a specific position in doubly linked list, we can use the following steps:

- Traverse to the node at the specified position, say **curr**.
- If the position is valid, adjust the pointers to skip the node to be deleted.
    - If curr is not the head of the linked list, update the next pointer of the node before curr to point to the node after curr, **curr->prev->next = curr-next**.
    - If curr is not the last node of the linked list, update the previous pointer of the node after curr to the node before curr, **curr->next->prev = curr->prev**.
- Free the memory allocated for the deleted node.

Below is the implementation of the above approach:

```
// Function to delete a node at a specific
//position in the doubly linked list
struct Node * delPos(struct Node * head, int pos)
{
    // If the list is empty
    if (head == NULL)
        return head;
    struct Node * curr = head;
    // Traverse to the node at the given position
    for (int i = 1; curr && i < pos; ++i) {
        curr = curr -> next;
    }
   // If the position is out of range
    if (curr == NULL)
       return head;
    // Update the previous node's next pointer
    if (curr -> prev)
        curr -> prev -> next = curr -> next;
    // Update the next node's prev pointer
    if (curr -> next)
        curr -> next -> prev = curr -> prev;
    // If the node to be deleted is the head node
    if (head == curr)
        head = curr -> next;
    // Deallocate memory for the deleted node
    free(curr);
    return head;
}
```

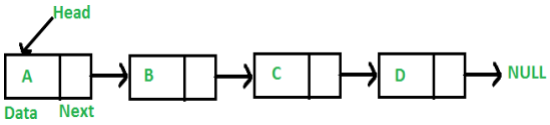
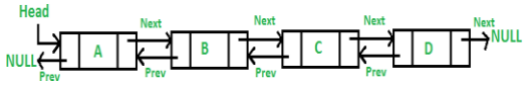
Advantages of Doubly Linked List

- **Efficient traversal in both directions:** Doubly linked lists allow for efficient traversal of the list in both directions, making it suitable for applications where frequent insertions and deletions are required.
- **Easy insertion and deletion of nodes:** The presence of pointers to both the previous and next nodes makes it easy to insert or delete nodes from the list, without having to traverse the entire list.
- **Can be used to implement a stack or queue:** Doubly linked lists can be used to implement both stacks and queues, which are common data structures used in programming.

Disadvantages of Doubly Linked List

- **More complex than singly linked lists:** Doubly linked lists are more complex than singly linked lists, as they require additional pointers for each node.
- **More memory overhead:** Doubly linked lists require more memory overhead than singly linked lists, as each node stores two pointers instead of one.

Difference between Singly linked list and Doubly linked list

Singly linked list (SLL)	Doubly linked list (DLL)
SLL nodes contains 2 field -data field and next link field.	DLL nodes contains 3 fields -data field, a previous link field and a next link field.
	
In SLL, the traversal can be done using the next node link only. Thus traversal is possible in one direction only.	In DLL, the traversal can be done using the previous node link or the next node link. Thus traversal is possible in both directions (forward and backward).
Supports lesser number of operations in constant time	Supports additional operations like insert before, delete previous, delete current node and delete last in O(1) time. Since it supports delete last, it is used to efficiently implement Deque.
The SLL occupies less memory than DLL as it has only 2 fields.	The DLL occupies more memory than SLL as it has 3 fields.
Complexity of deletion with a given node is O(n), because the previous node needs to be known, and traversal takes O(n)	Complexity of deletion with a given node is O(1) because the previous node can be accessed easily
A singly linked list consumes less memory as compared to the doubly linked list.	The doubly linked list consumes more memory as compared to the singly linked list.
Singly linked list is relatively less used in practice due to limited number of operations	Doubly linked list is implemented more in libraries due to wider number of operations. For example Java LinkedList implements Doubly Linked List.

Introduction to Circular Linked List

A **circular linked list** is a data structure where the last node connects back to the first, forming a loop. This structure allows for continuous traversal without any interruptions. Circular linked lists are especially helpful for tasks like **scheduling** and **managing playlists,** this allowing for smooth navigation. In this tutorial, we'll cover the basics of circular linked lists, how to work with them, their advantages and disadvantages, and their applications.

What is a Circular Linked List?

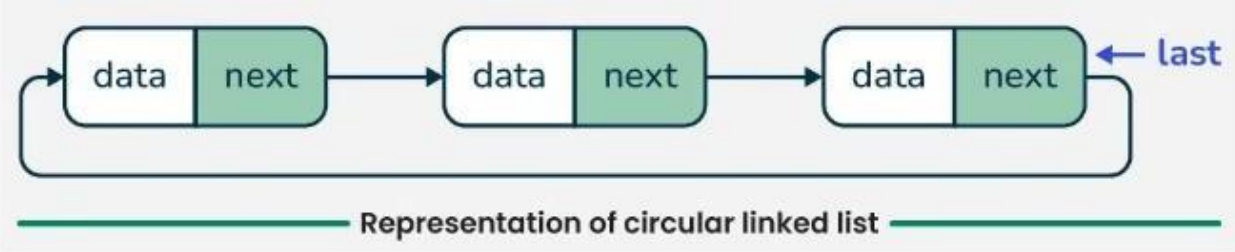
A **circular linked list** is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to **NULL**, the last node in a circular linked list points back to the first node. This means that you can keep traversing the list without ever reaching a **NULL** value.

Types of Circular Linked Lists

We can create a circular linked list from both singly linked lists and doubly linked lists. So, circular linked list are basically of two types:

1. Circular Singly Linked List:

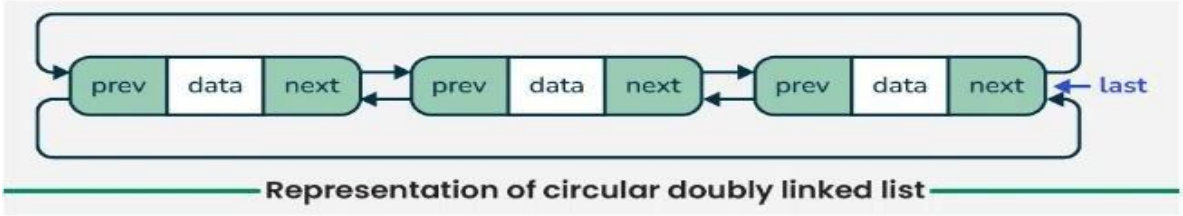
In **Circular Singly Linked List**, each node has just one pointer called the "**next**" pointer. The next pointer of **last node** points back to the **first node** and this results in forming a circle. In this type of Linked list we can only move through the list in one direction.



Representation of Circular Singly Linked List

2. Circular Doubly Linked List:

In **circular doubly linked list,** each node has two pointers **prev** and **next,** similar to doubly linked list. The **prev** pointer points to the previous node and the **next** points to the next node. Here, in addition to the **last** node storing the address of the first node, the **first node** will also store the address of the **last node**.

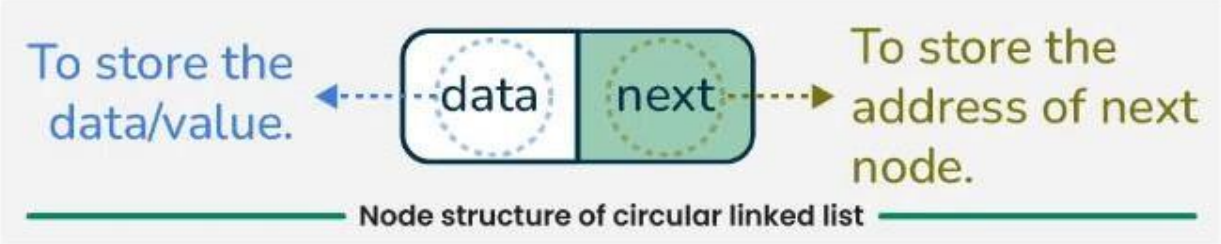


Representation of Circular Doubly Linked List

Note: In this article, we will use the circular singly linked list to explain the working of circular linked lists.

Representation of a Circular Singly Linked List

Let's take a look on the structure of a circular linked list.



Representation of a Circular Singly Linked List

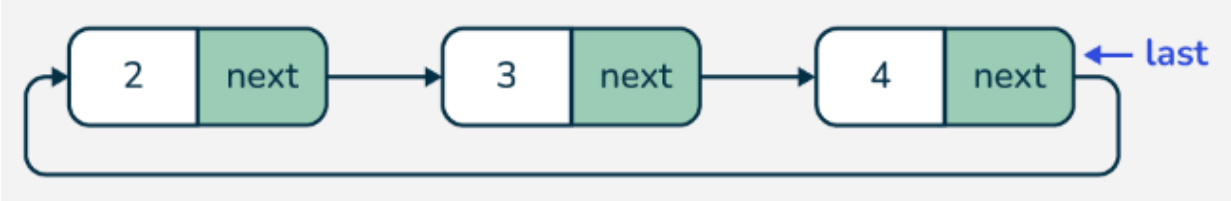
Create/Declare a Node of Circular Linked List

Syntax to Declare a Circular Linked List in Different Languages:

```
// Node structure
struct Node
{
   int data;
   struct Node *next;
};
// Function to create a new node
struct Node *createNode(int value)
{
   // Allocate memory
   struct Node *newNode =
    (struct Node *)malloc(sizeof(struct Node));
   // Set the data
   newNode->data = value;
   // Initialize next to NULL
   newNode->next = NULL;
   // Return the new node
   return newNode;
}
```

Example of Creating a Circular Linked List

Here's an example of creating a circular linked list with three nodes (2, 3, 4):



Created a circular linked list with 3 nodes

```
// Allocate memory for nodes
struct Node *first =  (struct Node *)malloc(sizeof(struct Node));
struct Node *second = (struct Node *)malloc(sizeof(struct Node));
struct Node *last =  (struct Node *)malloc(sizeof(struct Node));
// Initilize nodes
first->data = 2;
second->data = 3;
last->data = 4;
// Connect nodes
first->next = second;
second->next = last;
last->next = first;
```

In the above code, we have created three nodes **first, second,** and **last** having values **2, 3,** and **4** respectively.

- After creating three nodes, we have connected these node in a series.
- Connect the first node "**first"** to "**second"** node by **s**toring the address of "**second"** node into **first's** next
- Connect the second node "**second"** to "**second"** node by **s**toring the address of "**third**" node into **second's** next
- After connecting all the nodes, we reach the key characteristic of a circular linked list: linking the last node back to the first node. Therefore, we store the address of the "**first**" node in the "**last**" node.

Operations on the Circular Linked list:

We can do some operations on the circular linked list similar to the singly and doubly linked list which are:

- **Insertion**
  - Insertion at the empty list
  - Insertion at the beginning
  - Insertion at the end
  - Insertion at the given position
- **Deletion**

- Delete the first node
- Delete the last node
- Delete the node from any position

- **Searching**

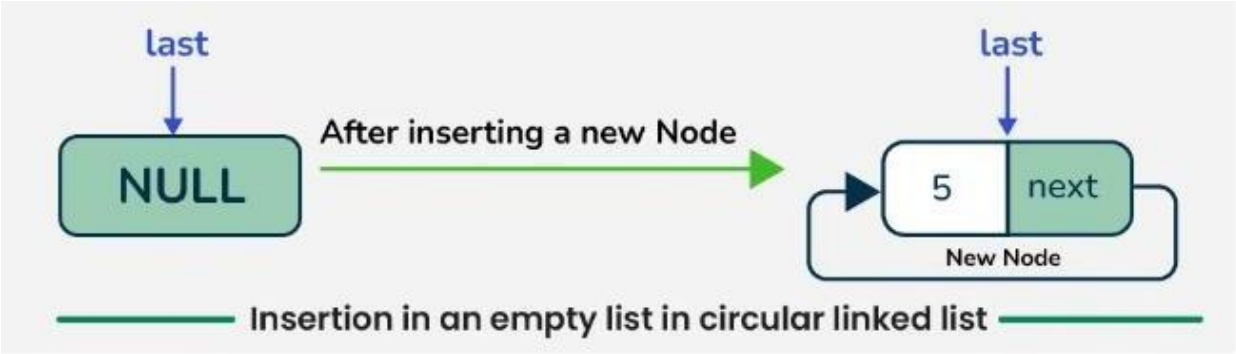
Note: We will be using the circular singly linked list to represent the working of the circular linked list.

1. Insertion in the circular linked list:

Insertion is a fundamental operation in linked lists that involves adding a new node to the list. The only extra step is connecting the last node to the first one. In the circular linked list mentioned below, we can insert nodes in four ways:

a. Insertion in an empty List in the circular linked list

*To insert a node in empty circular linked list, creates a **new node** with the given data, sets its next pointer to point to itself, and updates the **last** pointer to reference this **new node**.*

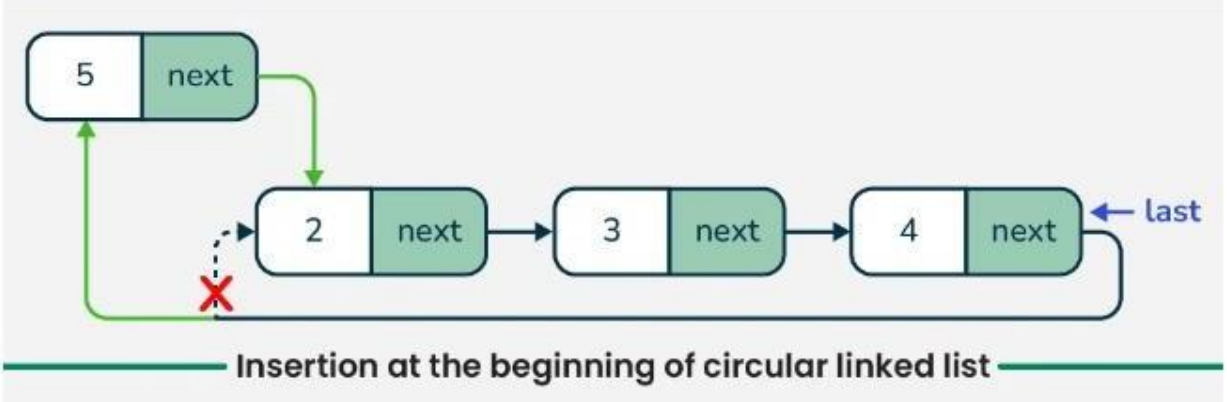


Insertion in an empty List

```
// Function to insert a node into an empty
// circular singly linked list
struct Node* insertInEmptyList(struct Node* last, int data)
{
   if (last != NULL) return last;

   // Create a new node
   struct Node* newNode = createNode(data);

   // Update last to point to the new node
   last = newNode;
   return last;
}
```

b. Insertion at the beginning in circular linked list

*To insert a new node at the beginning of a circular linked list, we first create the **new node** and allocate memory for it. If the list is empty (indicated by the last pointer being **NULL**), we make the **new node** point to itself. If the list already contains nodes then we set the **new node's** next pointer to point to the **current head** of the list (which is **last->next**), and then update the last node's next pointer to point to the **new node**. This maintains the circular structure of the list.*



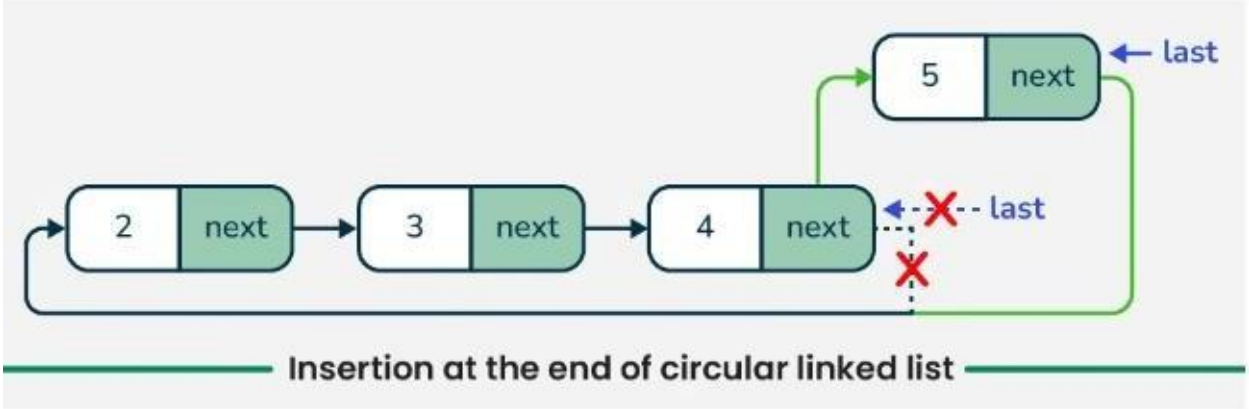
Insertion at the beginning in circular linked list

```
// Function to insert a node at the beginning
// of the circular linked list
struct Node *insertAtBeginning(struct Node *last, int value)
{
   struct Node *newNode = createNode(value);
   // If the list is empty, make the new node point to itself
   // and set it as last
   if (last == NULL)
   {
      newNode->next = newNode;
      return newNode;
   }
   // Insert the new node at the beginning
   newNode->next = last->next;
   last->next = newNode;
   return last;
}
```

c. Insertion at the end in circular linked list

*To insert a new node at the end of a circular linked list, we first create the new node and allocate memory for it. If the list is empty (mean, **last** or **tail** pointer being **NULL**), we initialize the list with the **new node** and making it point to itself to form a circular structure. If the list*

*already contains nodes then we set the **new node's** next pointer to point to the **current head** (which is **tail->next**), then update the **current tail's** next pointer to point to the **new node**. Finally, we update the **tail pointer** to the **new node.** This will ensure that the **new node** is now the **last node** in the list while maintaining the circular linkage.*



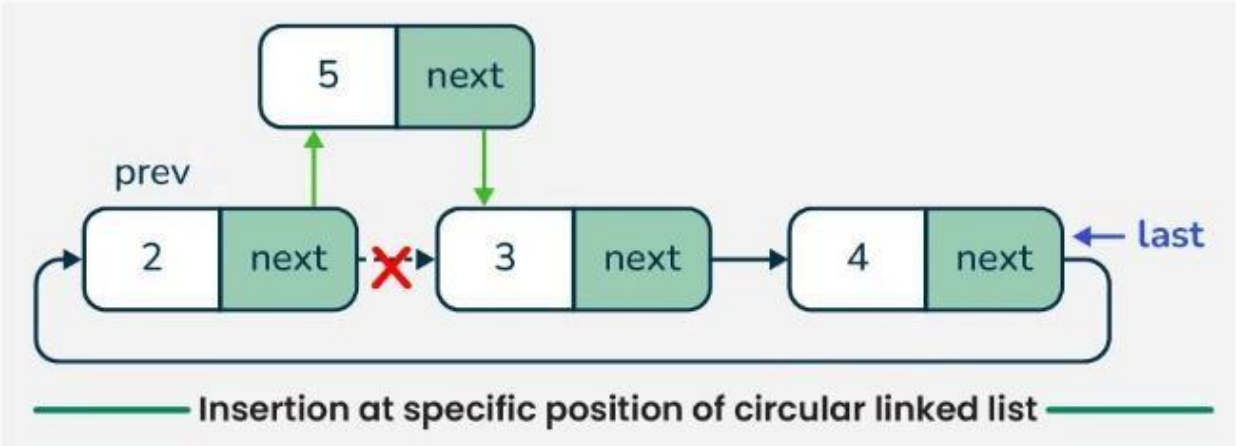
Insertion at the end in circular linked list

```
// Function to insert a node at the end of a circular linked list
struct Node *insertEnd(struct Node *tail, int value)
{
   struct Node *newNode = createNode(value);
   if (tail == NULL)
   {
      // If the list is empty, initialize it with the new node
      tail = newNode;
      newNode->next = newNode;
   }
   else
   {
      // Insert new node after the current tail and update the tail pointer
      newNode->next = tail->next;
      tail->next = newNode;
      tail = newNode;
   }
   return tail;
}
```

d. Insertion at specific position in circular linked list

*To insert a new node at a specific position in a circular linked list, we first check if the list is empty. If it is and the **position** is not **1** then we print an error message because the position doesn't exist in the list. If the **position** is **1** then we create the **new node** and make it point to itself. If the list is not empty, we create the **new node** and traverse the list to find the correct insertion point. If the **position** is **1**, we insert the **new node** at the beginning by adjusting the*

*pointers accordingly. For other positions, we traverse through the list until we reach the desired position and inserting the **new node** by updating the pointers. If the new node is inserted at the end, we also update the **last** pointer to reference the new node, maintaining the circular structure of the list.*



Insertion at specific position in circular linked list

```
// Function to insert a node at a specific position in a circular linked list
struct Node* insertAtPosition(struct Node *last, int data, int pos) {
   if (last == NULL) {
      // If the list is empty
      if (pos != 1) {
         printf("Invalid position!\n");
         return last;
      }
      // Create a new node and make it point to itself
      struct Node *newNode = createNode(data);
      last = newNode;
      last->next = last;
      return last;
   }
   // Create a new node with the given data
   struct Node *newNode = createNode(data);
   // curr will point to head initially
   struct Node *curr = last->next;
   if (pos == 1)
  {
      // Insert at the beginning
      newNode->next = curr;
      last->next = newNode;
      return last;
   }
```

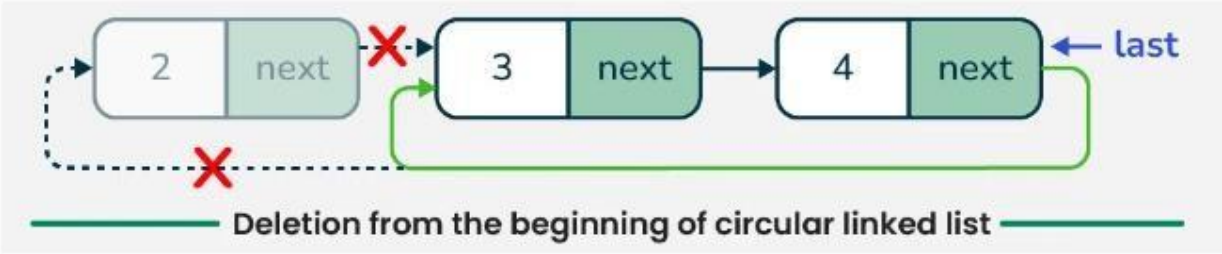
```
    // Traverse the list to find the insertion point
    for (int i = 1; i < pos - 1; ++i) {
        curr = curr->next;
        // If position is out of bounds
        if (curr == last->next) {
            printf("Invalid position!\n");
            return last;
        }
    }
    // Insert the new node at the desired position
    newNode->next = curr->next;
    curr->next = newNode;
    // Update last if the new node is inserted at the end
    if (curr == last) last = newNode;
    return last;
}
```

2. Deletion from a Circular Linked List

Deletion involves removing a node from the linked list. The main difference is that we need to ensure the list remains circular after the deletion. We can delete a node in a circular linked list in three ways:

a. Delete the first node in circular linked list

*To delete the first node of a circular linked list, we first check if the list is empty. If it is then we print a message and return **NULL**. If the list contains only one node (the **head** is the same as the **last**) then we delete that node and set the **last** pointer to **NULL**. If there are multiple nodes then we update the **last->next** pointer to skip the **head** node and effectively removing it from the list. We then delete the **head** node to free the allocated memory. Finally, we return the updated **last** pointer, which still points to the **last** node in the list.*



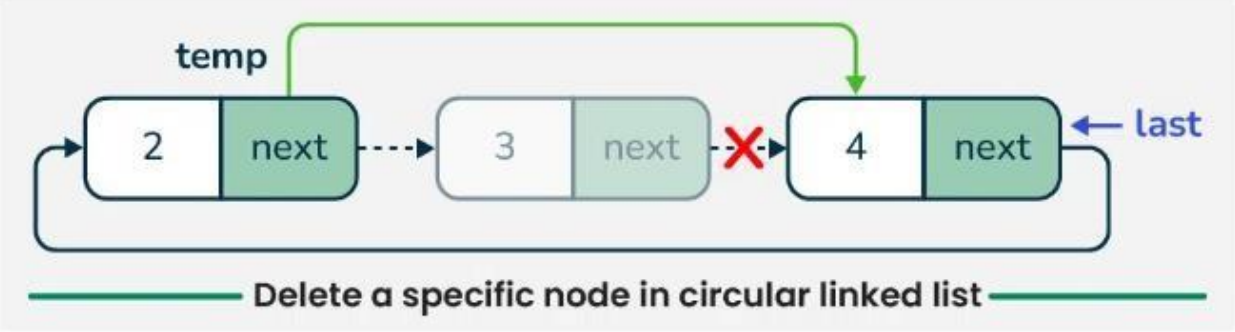
Delete the first node in circular linked list

```
struct Node* deleteFirstNode(struct Node* last)
{
    if (last == NULL)
    {
```

```
    // If the list is empty
    printf("List is empty\n");
    return NULL;
  }
  struct Node* head = last->next;
  if (head == last)
  {
    // If there is only one node in the list
    free(head);
    last = NULL;
  }
  else
  {
    // More than one node in the list
    last->next = head->next;
    free(head);
  }
  return last;
}
```

b. Delete a specific node in circular linked list

*To delete a specific node from a circular linked list, we first check if the list is empty. If it is then we print a message and return **nullptr**. If the list contains only one node and it matches the **key** then we delete that node and set **last** to **nullptr**. If the node to be deleted is the first node then we update the **next** pointer of the **last** node to skip the **head** node and delete the **head**. For other nodes, we traverse the list using two pointers: **curr** (to find the node) and **prev** (to keep track of the previous node). If we find the node with the matching key then we update the next pointer of **prev** to skip the **curr** node and delete it. If the node is found and it is the last node, we update the **last** pointer accordingly. If the node is not found then do nothing and **tail** or **last** as it is. Finally, we return the updated **last** pointer.*



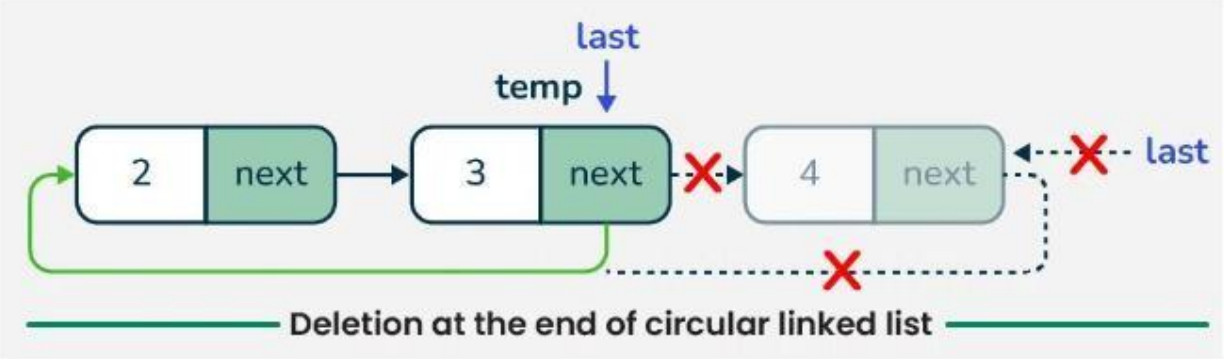
Delete a specific node in circular linked list

```
// Function to delete a specific node in the circular linked list
struct Node* deleteSpecificNode(struct Node* last, int key)
```

```
{
   if (last == NULL)
   {
      // If the list is empty
      printf("List is empty, nothing to delete.\n");
      return NULL;
   }
   struct Node* curr = last->next;
   struct Node* prev = last;
   // If the node to be deleted is the only node in the list
   if (curr == last && curr->data == key)
  {
      free(curr);
      last = NULL;
      return last;
   }
   // If the node to be deleted is the first node
   if (curr->data == key)
   {
      last->next = curr->next;
      free(curr);
      return last;
   }
   // Traverse the list to find the node to be deleted
   while (curr != last && curr->data != key) {
      prev = curr;
      curr = curr->next;
   }
   // If the node to be deleted is found
   if (curr->data == key)
   {
      prev->next = curr->next;
      if (curr == last)
      {
         last = prev;
      }
      free(curr);
   } else {
      // If the node to be deleted is not found
      printf("Node with data %d not found.\n", key);
   }
   return last;
}
```

C. Deletion at the end of Circular linked list

*To delete the last node in a circular linked list, we first check if the list is empty. If it is, we print a message and return **nullptr**. If the list contains only one node (where the **head** is the same as the **last**), we delete that node and set **last** to **nullptr**. For lists with multiple nodes, we need to traverse the list to find the **second last node**. We do this by starting from the **head** and moving through the list until we reach the node whose next pointer points to **last**. Once we find the **second last** node then we update its next pointer to point back to the **head,** this effectively removing the last node from the list. We then delete the last node to free up memory and return the updated **last** pointer, which now points to the last node.*



Deletion at the end of Circular linked list

```
// Function to delete the last node in the circular linked list
struct Node* deleteLastNode(struct Node* last)
{
   if (last == NULL)
  {
      // If the list is empty
      printf("List is empty, nothing to delete.\n");
      return NULL;
   }
   struct Node* head = last->next;
   // If there is only one node in the list
   if (head == last)
  {
      free(last);
      last = NULL;
      return last;
   }
   // Traverse the list to find the second last node
   struct Node* curr = head;
   while (curr->next != last)
  {
      curr = curr->next;
```

```
    }
    // Update the second last node's next pointer to point to head
    curr->next = head;
    free(last);
    last = curr;
    return last;
}
```

3. Searching in Circular Linked list

Searching in a circular linked list is similar to searching in a regular linked list. We start at a given node and traverse the list until you either find the target value or return to the starting node. Since the list is circular, make sure to keep track of where you started to avoid an infinite loop.

*To search for a specific value in a circular linked list, we first check if the list is empty. If it is then we return **false**. If the list contains nodes then we start from the **head** node (which is the **last->next**) and traverse the list. We use a pointer **curr** to iterate through the nodes until we reach back to the **head**. During traversal, if we find a node whose **data** matches the given **key** then we return **true** to indicating that the value was found. After the loop, we also check the last node to ensure we don't miss it. If the **key** is not found after traversing the entire list then we return **false**.*

```
// Function to search for a specific value in the circular linked list
int search(struct Node *last, int key){
    if (last == NULL){
        // If the list is empty
        return 0;
    }
    struct Node *head = last->next;
    struct Node *curr = head;
    // Traverse the list to find the key
    while (curr != last){
        if (curr->data == key)
        {
            // Key found
            return 1;
        }
        curr = curr->next;
    }
    // Check the last node
    if (last->data == key)
    {
        // Key found
```

```
        return 1;
    }
    // Key not found
    return 0;
}
```

Advantages of Circular Linked Lists

- In circular linked list, the last node points to the first node. There are no null references, making traversal easier and reducing the chances of encountering null pointer exceptions.
- We can traverse the list from any node and return to it without needing to restart from the head, which is useful in applications requiring a circular iteration.
- Circular linked lists can easily implement circular queues, where the last element connects back to the first, allowing for efficient resource management.
- In a circular linked list, each node has a reference to the next node in the sequence. Although it doesn't have a direct reference to the previous node like a doubly linked list, we can still find the previous node by traversing the list.

Disadvantages of Circular Linked Lists

- Circular linked lists are more complex to implement than singly linked lists.
- Traversing a circular linked list without a clear stopping condition can lead to infinite loops if not handled carefully.
- Debugging can be more challenging due to the circular nature, as traditional methods of traversing linked lists may not apply.